Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 5:

The File System with Your Own Hands

Elaborated: st. gr. FAF-243
Șoimu Ionuț

Verified: asist. univ.
Reițman Patricia

Chișinău – 2026

Contents

1 Lab 5: Track A – The File System with Your Own Hands **2**
1.1 Before you start . . . 2
1.2 Part 1. The tree and the file types . . . 2
1.2.1 Hierarchical Tree Navigation and Type Inspection . . . 2
1.3 Part 2. Inodes and links . . . 3
1.3.1 Inodes, Hard Links, and Unlinking . . . 3
1.3.2 Symbolic Links and Dangling References . . . 4
1.3.3 Directory Link Counts and Sparse Files . . . 4
1.4 Part 3. Permissions . . . 6
1.4.1 File and Directory Permission Tests . . . 6
1.5 Part 4. Disks and the kernel as files . . . 8
1.5.1 Storage Devices, Mount Tables, and /proc Introspection . . . 8
1.6 Stretch Goals (Optional) . . . 9

2 Conclusion **10**

1 Lab 5: Track A – The File System with Your Own Hands

1.1 Before you start

Purpose and Scope

The filesystem is one of the foundational abstractions provided by modern operating systems. The purpose of this laboratory work is to inspect and verify the physical and logical architecture of the Linux filesystem directly on bare-metal hardware.

We explore the unified directory tree and POSIX file classifications, investigate the fundamental separation between filesystem names and underlying inode structures, examine hard links versus symbolic links, analyze directory link-count mechanics, evaluate apparent file sizes against physical block allocations (sparse files), and dissect the multi-tiered permission model (discretionary access control, umask arithmetic, directory traversal rules, and special bits such as SetUID and Sticky Bit). Finally, we inspect how storage disks, partitions, and runtime kernel objects are surfaced through virtual pseudo-filesystems.

Environment Setup and User Identification

As instructed, all experiments are performed as an unprivileged standard user (`ionut`) rather than `root`, ensuring that kernel access-control checks and permission boundaries are strictly enforced.

```
➜  ~ whoami
ionut
➜  ~ uname -a
Linux arch 7.2.9-2-cachyos #1 SMP PREEMPT_DYNAMIC Fri, 09 Oct 2026 15:36:46 +0000
    x86_64 GNU/Linux
➜  ~ uptime
 15:56:40 up  2:42,  1 user,  load average: 0.67, 0.64, 0.82
➜  ~ mkdir -p ~/os-lab5 && cd ~/os-lab5
```

1.2 Part 1. The tree and the file types

1.2.1 Hierarchical Tree Navigation and Type Inspection

We inspect the root directory and examine representative filesystem entries using `ls -ld`, `file`, and du:

```
➜  os-lab5 ls /
bin   dev  home  lib64  opt   root  sbin  sys  usr
boot  etc  lib   mnt    proc  run   srv   tmp  var

➜  os-lab5 ls -ld /etc /etc/passwd /bin /dev/null /tmp
drwxr-xr-x  1 root root 3862 Oct 12 15:29 /etc
-rw-r--r--  1 root root 2182 Jun 25 14:34 /etc/passwd
lrwxrwxrwx  1 root root    7 Oct 12  2025 /bin -> usr/bin
crw-rw-rw-  1 root root 1, 3 Oct 12 13:14 /dev/null
drwxrwxrwt 18 root root  440 Oct 12 15:55 /tmp

➜  os-lab5 ls -l /dev | head -10
total 0
crw-r--r--   1 root  root    10,   235 Oct 12 13:14 autofs
drwxr-xr-x   2 root  root          240 Oct 12 13:14 block
crw-rw----   1 root  disk    10,   234 Oct 12 13:14 btrfs-control
```

```
drwxr-xr-x   3 root  root           60 Oct 12 13:14 bus
drwxr-xr-x   2 root  root         5240 Oct 12 15:17 char
crw-------   1 root  tty      5,     1 Oct 12 13:14 console
lrwxrwxrwx   1 root  root           11 Oct 12 13:14 core -> /proc/kcore
drwxr-xr-x  14 root  root          280 Oct 12 13:14 cpu
crw-------   1 root  root    10,   259 Oct 12 13:14 cpu_dma_latency

➜  os-lab5 file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
    linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=
    d572a7b5009dbbcb275c2e516be58e706ae248a9, for GNU/Linux 4.4.0, stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin

➜  os-lab5 du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
1.8G    /var/log
24G     /usr
```

Observe:

1. **For each path in the second line, write its type from the first letter of ls -l.**
   - `/etc`: `d` – Directory (a container holding mapping pairs of names to inodes).
   - `/etc/passwd`: `-` – Regular file (standard sequential byte stream on disk).
   - `/bin`: `l` – Symbolic link (a file whose content is a path string to another target).
   - `/dev/null`: `c` – Character device (a stream-oriented hardware/kernel device driver).
   - `/tmp`: `d` – Directory (with sticky bit `t` set).

2. **What is /bin on your system: a directory or a link, and to where?**
   On this system, `/bin` is a **symbolic link** pointing to `usr/bin` (`/bin -> usr/bin`). This reflects modern Linux distributions' adoption of the **UsrMerge** architecture, which consolidates all essential system binaries into `/usr/bin` while preserving backward compatibility via symlinks at the root level.

3. **Find one character device and one block device in /dev.**
   - **Character device:** `/dev/null` (major 1, minor 3) or `/dev/console` (major 5, minor 1) or `/dev/autofs`.
   - **Block device:** `/dev/nvme1n1p2` (major 259, minor 8) or `/dev/zram0` (major 253, minor 0), representing block-addressable persistent or memory storage.

4. **Which is bigger on your laptop, /var/log or /usr, and by roughly how much?**
   `/usr` (24 GB) is substantially larger than `/var/log` (1.8 GB) by approximately **22.2 GB** (more than 13 times larger). This is expected because `/usr` houses all installed software binaries, libraries, header files, and shared static data, whereas `/var/log` contains only system telemetry and service logs.

1.3 Part 2. Inodes and links

1.3.1 Inodes, Hard Links, and Unlinking

We create a file, inspect its inode metadata using `stat` and `ls -li`, create a hard link, remove the initial directory entry, and verify data accessibility:

```
➜  os-lab5 echo hello > notes.txt
➜  os-lab5 stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 0,47    Inode: 10245582    Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   ionut)   Gid: ( 1000/   ionut)
Access: 2026-10-12 15:57:54.278555788 +0300
Modify: 2026-10-12 15:57:54.278555788 +0300
Change: 2026-10-12 15:57:54.278555788 +0300
 Birth: 2026-10-12 15:57:54.278555788 +0300

➜  os-lab5 ls -li notes.txt
10245582 -rw-r--r-- 1 ionut ionut 6 Oct 12 15:57 notes.txt

➜  os-lab5 ln notes.txt hard.txt
➜  os-lab5 ls -li notes.txt hard.txt
10245582 -rw-r--r-- 2 ionut ionut 6 Oct 12 15:57 hard.txt
10245582 -rw-r--r-- 2 ionut ionut 6 Oct 12 15:57 notes.txt

➜  os-lab5 rm notes.txt
➜  os-lab5 cat hard.txt ; ls -li hard.txt
hello
10245582 -rw-r--r-- 1 ionut ionut 6 Oct 12 15:57 hard.txt
```

1.3.2 Symbolic Links and Dangling References

We create a soft (symbolic) link pointing to `hard.txt`, inspect inode numbers, remove the target, and observe the broken reference:

```
➜  os-lab5 ln -s hard.txt soft.txt
➜  os-lab5 ls -li hard.txt soft.txt
10245582 -rw-r--r-- 1 ionut ionut 6 Oct 12 15:57 hard.txt
10245583 lrwxrwxrwx 1 ionut ionut 8 Oct 12 15:57 soft.txt -> hard.txt

➜  os-lab5 cat soft.txt
hello

➜  os-lab5 rm hard.txt
➜  os-lab5 cat soft.txt
cat: soft.txt: No such file or directory

➜  os-lab5 ls -l soft.txt
lrwxrwxrwx 1 ionut ionut 8 Oct 12 15:57 soft.txt -> hard.txt
```

1.3.3 Directory Link Counts and Sparse Files

We examine directory link dynamics and compare apparent byte size against allocated disk space:

```
➜  os-lab5 mkdir d ; ls -ld d
drwxr-xr-x 1 ionut ionut 0 Oct 12 15:58 d
➜  os-lab5 mkdir d/x d/y ; ls -ld d
drwxr-xr-x 1 ionut ionut 4 Oct 12 15:58 d

➜  os-lab5 echo x > one.txt ; ls -l one.txt ; du -h one.txt
```

```
-rw-r--r-- 1 ionut ionut 2 Oct 12 15:58 one.txt
4.0K    one.txt

➜  os-lab5 truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-r--r-- 1 ionut ionut 1.0G Oct 12 15:58 big.img
0       big.img
➜  os-lab5 rm big.img
```

Observe:

1. **Which number in stat is the inode, and which tells how many names the file has?**
   In the output of `stat`, the field `Inode:` gives the inode number (here `10245582`), and the field `Links:` indicates the reference count of directory entries pointing to that inode (`Links: 1`, increasing to `Links: 2` after `ln`).

2. **After rm notes.txt, why could you still read hard.txt? What does rm really remove?**
   In Unix filesystems, a file's metadata and data blocks are owned by its **inode**, not by any directory name. A hard link is simply an additional directory entry pointing directly to that exact same inode number. When you run `rm notes.txt`, the system executes the `unlink()` system call: it removes only the directory entry named `notes.txt` from the folder and decrements the inode's `nlink` counter from 2 to 1. Because `hard.txt` still references inode 10245582 and `nlink` remains > 0, the inode and its underlying disk blocks are preserved intact.

3. **Why did cat soft.txt fail at the end, although soft.txt still exists? How big is soft.txt and why exactly that size?**
   A symbolic link does not point to an inode; its data payload consists solely of a pathname string (`"hard.txt"`). When `cat soft.txt` runs, the kernel follows the link and attempts to open the file named by that target string. Because `hard.txt` was removed, the lookup fails with `No such file or directory` (a dangling symlink).
   `soft.txt` has a reported size of exactly **8 bytes** because the target string `"hard.txt"` contains exactly 8 ASCII characters. In symbolic links, the file size is always equal to the character length of the target destination path.

4. **A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.**
   In classic Unix filesystems (such as ext4), directory links arise from:

   - For a newly created directory `d`:
     (a) Link 1: The entry named `d` inside the parent directory.
     (b) Link 2: The self-referential entry `.` inside `d` itself.
   - After creating subdirectories `d/x` and `d/y`:
     (c) Link 3: The parent reference `..` inside `d/x`.
     (d) Link 4: The parent reference `..` inside `d/y`.

   Each child subdirectory contains a `..` entry pointing back to `d`, incrementing `d`'s link count by 1. *(Note: modern btrfs filesystems report internal sub-item counts differently, displaying 0 initially and 4 upon subfolder creation)*

5. **Why does a 2-byte file use 4.0K, and a 1 GB file use 0?**

Filesystems allocate persistent storage in discrete chunks called **disk blocks** (typically 4096 bytes / 4.0 KB). Even though `one.txt` contains only 2 bytes of payload (`'x'` and `'\n'`), the kernel must allocate at least one full 4.0 KB block to house it.
Conversely, `truncate -s 1G big.img` creates a **sparse file**. The kernel updates the file's inode metadata to record an apparent logical length of 1 GB, but allocates zero physical storage blocks on disk. The unwritten span is treated as a "hole"; reads return zero-filled buffers on demand, and `du` accurately reports 0 allocated disk usage.

1.4 Part 3. Permissions

1.4.1 File and Directory Permission Tests

We explore permission masks, umask effects, directory read/execute semantics, and special bits:

```
➜  os-lab5 echo secret > s.txt
➜  os-lab5 chmod 600 s.txt ; ls -l s.txt
-rw------- 1 ionut ionut 7 Oct 12 15:59 s.txt

➜  os-lab5 chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

➜  os-lab5 chmod u+r s.txt ; cat s.txt
secret

➜  os-lab5 umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0022
-rw-r--r-- 1 ionut ionut 0 Oct 12 15:59 new.txt
drwxr-xr-x 1 ionut ionut 0 Oct 12 15:59 newdir

➜  os-lab5 mkdir box ; echo hi > box/f
➜  os-lab5 chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied

➜  os-lab5 chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi

➜  os-lab5 chmod 755 box
➜  os-lab5 ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x  1 root root  88984 Jul 31 22:01 /usr/bin/passwd
drwxrwxrwt 17 root root    420 Oct 12 15:58 /tmp

➜  os-lab5 cat /etc/shadow
cat: /etc/shadow: Permission denied
➜  os-lab5 ls -l /etc/shadow
-rw------- 1 root root 1046 Jun 25 14:34 /etc/shadow
```

Observe:

1. **Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.**

   - **600** = `-rw-------`: Owner read/write, no access for anyone else. Used for confidential private keys (e.g., `~/.ssh/id_rsa`, `~/.ssh/id_ed25519`) and sensitive personal configuration files.

- **644** = `-rw-r--r--`: Owner read/write, group and others read-only. Standard for public text files, system configuration files (e.g., `/etc/passwd`), and static web assets.
- **755** = `-rwxr-xr-x`: Owner full access, group and others read and execute. Standard for executable system binaries (e.g., `/usr/bin/ls`) and publicly traversable directories.
- **700** = `drwx------`: Owner full access, no access for group or others. Used for private user directories requiring strict isolation (e.g., `~/.ssh`, `~/.gnupg`, root's `/root`).

2. **With chmod 000 you own the file but cannot read it. Why does ownership not help?**
   Linux evaluates discretionary access permissions according to a strict hierarchical priority check:

   1. Is UID equal to file owner? $\longrightarrow$ Apply owner bits only. Do not check group or others.

   When `chmod 000` is set, the owner permission triple is `---` (0). Because the accessing user is the file owner, the kernel checks the owner bits, detects that read permission is explicitly absent, and immediately rejects the `open()` request with `EACCES` (*Permission denied*). Ownership grants the right to alter permissions via `chmod`, but does not grant automatic read/write access unless owner permission bits are set.

3. **What is your umask, and how does it explain the permissions of new.txt and newdir?**
   The active umask is `0022` (octal).
   When creating files and directories, the operating system applies the umask as a bitwise exclusion filter:

   - **Files:** Base mode is `0666` (`rw-rw-rw-`). Applying the umask:

   $$0666 \;\&\; \sim(0022) = 0644 \quad (\texttt{-rw-r--r--})$$

   - **Directories:** Base mode is `0777` (`rwxrwxrwx`). Applying the umask:

   $$0777 \;\&\; \sim(0022) = 0755 \quad (\texttt{drwxr-xr-x})$$

   This prevents group and other users from having write permissions on newly generated files by default while keeping them readable.

4. **Explain the box experiment: what does r allow on a directory, and what does x allow?**
   On directories, `r` and `x` serve fundamentally distinct roles:

   - **Read (`r`):** Grants permission to read the directory's raw entry list (names of items). With `chmod u=r`, `ls box` succeeds and lists filename `f`. However, without `x`, the kernel cannot traverse into the directory to resolve `f`'s inode, so `cat box/f` fails with `Permission denied`.
   - **Execute (`x`):** Grants permission to traverse (search) through the directory. With `chmod u=x`, `ls box` fails with `Permission denied` because directory enumeration is prohibited; however, `cat box/f` succeeds and prints `hi` because the user knows the target path and has permission to traverse the directory to access the file.

5. **What do the s in /usr/bin/passwd and the t in /tmp mean, and why does passwd need the s?**

- **The `s` in `/usr/bin/passwd`** represents the **SetUID (Set User ID)** bit (`04000`). When executed, the process runs with the privileges of the file's owner (`root`) rather than the invoking user.
- **Why `passwd` needs `s`:** Standard users must be able to change their passwords, but password hashes are stored in `/etc/shadow`, which has permissions `0600` owned by `root`. An unprivileged user cannot write to `/etc/shadow`. By making `passwd` SetUID root, the program temporarily executes with root authority, validates the user's old credentials, updates `/etc/shadow`, and terminates safely.
- **The `t` in `/tmp`** represents the **Sticky Bit** (`01000`) on a directory. In a world-writable directory (`0777`), any user could delete any other user's files. The sticky bit enforces the restriction that only the file's owner (or `root`) may delete or rename a file within that directory.

1.5 Part 4. Disks and the kernel as files

1.5.1 Storage Devices, Mount Tables, and /proc Introspection

```
➜  os-lab5 lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
zram0       253:0    0     4G  0 disk [SWAP]
nvme0n1     259:0    0 476.9G  0 disk
|-nvme0n1p1 259:1    0   100M  0 part
|-nvme0n1p2 259:2    0    16M  0 part
|-nvme0n1p3 259:3    0  96.8G  0 part
|-nvme0n1p4 259:4    0   736M  0 part
`-nvme0n1p5 259:5    0 379.3G  0 part
nvme1n1     259:6    0 931.5G  0 disk
|-nvme1n1p1 259:7    0     1G  0 part /boot
`-nvme1n1p2 259:8    0 930.5G  0 part /home
                                      /var/log
                                      /var/cache/pacman/pkg
                                      /

➜  os-lab5 df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme1n1p2  931G  833G   94G  90% /

➜  os-lab5 mount | grep " / "
/dev/nvme1n1p2 on / type btrfs (rw,relatime,compress=zstd:3,ssd,discard=async,
    space_cache=v2,subvolid=256,subvol=/@)

➜  os-lab5 grep -c processor /proc/cpuinfo
12

➜  os-lab5 head -3 /proc/meminfo
MemTotal:       13326596 kB
MemFree:         1817992 kB
MemAvailable:    9386644 kB

➜  os-lab5 head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   9728
Ngid:   0
```

```
➜  os-lab5 ls -l /proc/self/fd
total 0
lrwx------ 1 ionut ionut 64 Oct 12 15:59 0 -> /dev/pts/1
lrwx------ 1 ionut ionut 64 Oct 12 15:59 1 -> /dev/pts/1
lrwx------ 1 ionut ionut 64 Oct 12 15:59 2 -> /dev/pts/1
lr-x------ 1 ionut ionut 64 Oct 12 15:59 3 -> /proc/9728/fd
```

Observe:

1. **Draw your disk: disk name, partitions, which one is mounted at / and with what file system type.**

Disk Device	Partition	Size	Filesystem	Mountpoint
nvme1n1 (931.5G)	nvme1n1p1	1.0 GB	vfat	/boot
	nvme1n1p2	**930.5 GB**	**btrfs** (zstd:3)	/, /home, /var/log
nvme0n1 (476.9G)	nvme0n1p1-p5	Multiple	NTFS/ext4	Unmounted / Secondary OS
zram0 (4.0G)	–	4.0 GB	swap	[SWAP]

Table 1: Storage topology: primary disk NVMe layout and mount configuration

2. **How full is your root file system?**
   The root filesystem (/dev/nvme1n1p2) is **90% full**, having used 833 GB out of 931 GB, with 94 GB of available free storage remaining.

3. **How many CPU cores and how much RAM does your laptop have, according to /proc?**

   - **CPU Cores: 12 logical cores** (grep -c processor /proc/cpuinfo).
   - **RAM: 13,326,596 kB** (≈ **13.3 GB**) total physical RAM, with 9.38 GB available for applications.

4. **/proc/self/status showed the name of a program. Which one, and why that one?**
   It showed Name: head.
   /proc/self is a magic symbolic link maintained dynamically by the Linux kernel. Whenever a process dereferences /proc/self, the kernel resolves it to the PID directory of the calling process itself (/proc/<PID>/). Because the command executed was head -5 /proc/self/status, the process opening and reading the file descriptor was head, so the kernel accurately reported Name: head.

1.6 Stretch Goals (Optional)

Experiment A: Magic Numbers and File Type Identification

We compress a text file using gzip and inspect the file's raw header bytes using file and xxd:

```
➜  os-lab5 gzip -c /etc/passwd > notes.gz
➜  os-lab5 file notes.gz
notes.gz: gzip compressed data, was "passwd", last modified: Thu Jun 25 11:34:06 2026,
    from Unix, original size modulo 2^32 2182
➜  os-lab5 xxd notes.gz | head -1
```

```
00000000: 1f8b 0808 2e12 3d6a 0003 7061 7373 7764  ......=j..passwd
```

Result: The first two bytes are `0x1F` and `0x8B` (`1f 8b`). These constitute the standardized **gzip magic number** defined in RFC 1952. The `file` utility reads these initial header bytes rather than relying on filename extensions to reliably identify file formats.

Experiment C: Deleting a File that is Still Open

We hold an open file descriptor on a file, remove its directory entry, and observe descriptor status in `/proc`:

```
➜  os-lab5 echo "data" > openfile.txt
➜  os-lab5 exec 3< openfile.txt
➜  os-lab5 rm openfile.txt
➜  os-lab5 ls -l /proc/$$/fd/3
lr-x------ 1 ionut ionut 64 Oct 12 15:59 /proc/9730/fd/3 -> '/home/ionut/os-lab5/
    openfile.txt (deleted)'
➜  os-lab5 exec 3<&-
```

Result: The symlink in `/proc/<PID>/fd/3` indicates `/home/ionut/os-lab5/openfile.txt (deleted)`. The directory entry is unlinked immediately, but the inode and data blocks remain allocated and accessible to the process. The physical storage space is only freed by the kernel when the last file descriptor referring to the inode is closed via `exec 3<&-` (or when the process exits).

2 Conclusion

Through this laboratory work, I explored the internal architecture and operational mechanisms of the Linux filesystem directly on a production system.

The experiments highlighted the architectural division between directory trees and inode metadata. By investigating hard links, I observed that filenames are merely directory pointers to an inode; unlinking a filename decrements the reference counter but leaves data blocks accessible until all links and open process descriptors are closed. In contrast, symbolic links operate as independent files storing pathname strings, exhibiting dangling behavior when their targets are removed.

Analyzing storage allocation demonstrated the distinction between apparent file size and physical disk blocks: small files consume a full block allocation unit (4.0 KB), whereas sparse files span gigabytes logically while consuming zero disk blocks. Examining permissions demystified discretionary access controls: owner bits take precedence regardless of user status, umask arithmetic enforces baseline privacy upon creation, directory read permissions govern name listing whereas execute permissions control path traversal, and special bits (SetUID and Sticky bit) enable secure privilege delegation and shared directory safety. Finally, introspecting ‘/proc‘ reinforced the Unix axiom that kernel state and devices are uniformly surfaced through file interfaces.